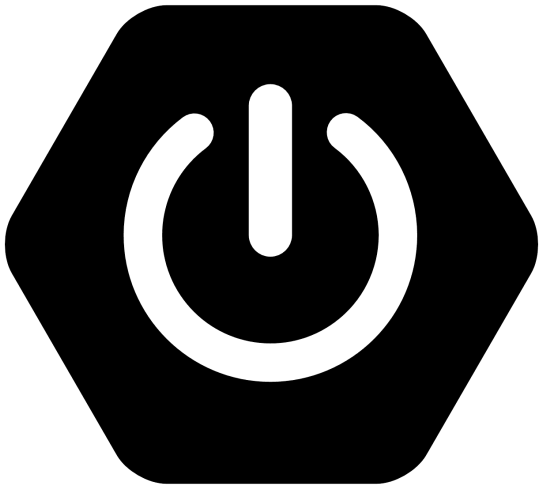
BNU

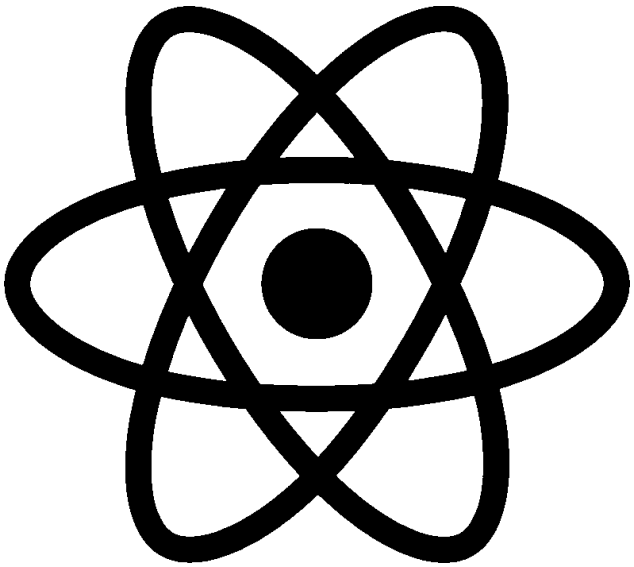
Saad Mughal
F2023-009
Total Marks: 100
21 November 2025

Web Engineering

Mid Term

Course Lecturer: Nouman Ali Shah
Section: A





Web Engineering (CSC324) - Mid Term

Expense Management System

SECTION B - CONCEPTUAL QUESTIONS

Q1. Write JPA relationship annotations for:

a) Members Entity

```java
package io.saadmughal.midterm.entities;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
@Entity
@Table(name = "members")
public class Member {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id", nullable = false)
    private Integer id;

    @Column(name = "first_name", length = 45)
    private String firstName;

    @Column(name = "email", length = 45)
    private String email;

}
```

b) Categories Entity

```java
package io.saadmughal.midterm.entities;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
@Entity
@Table(name = "categories")
public class Category {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id", nullable = false)
    private Integer id;
```

```
    @Column(name = "name", length = 45)
    private String name;

    @Column(name = "description", length = 45)
    private String description;

}
```

c) Expenses Entity

```
package io.saadmughal.midterm.entities;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;

@Getter
@Setter
@Entity
@Table(name = "expenses")
public class Expense {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id", nullable = false)
    private Integer id;

    @Column(name = "date", length = 45)
    private String date;

    @Column(name = "amount", precision = 10)
    private BigDecimal amount;

    @Column(name = "description", length = 200)
    private String description;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "Members_id", nullable = false)
    private Member members;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "categories_id", nullable = false)
    private Category categories;

}
```

Q2. Explain:

a) Difference between CrudRepository and JpaRepository

Feature	CrudRepository	JpaRepository

Functionality	Basic CRUD operations only	Extends CrudRepository + JPA-specific methods
Methods	save(), findById(), findAll(), deleteById()	All CRUD + flush(), saveAndFlush(), findAll(Sort), batch operations
Use Case	Simple applications	Most applications (recommended)

Example:

```
// CrudRepository
public interface MemberRepository extends CrudRepository<Member, Integer> {}

// JpaRepository (Better choice)
public interface MemberRepository extends JpaRepository<Member, Integer> {}
```

b) @RestController vs @Controller

Annotation	Purpose	Response Type	Use Case
@Controller	Traditional Spring MVC	Returns view (HTML/JSP)	Web pages with Thymeleaf/JSP
@RestController	REST API	Returns data (JSON/XML)	REST APIs, mobile backends

@RestController = @Controller + @ResponseBody

Example:

```
// Returns HTML view
@Controller
public class WebController {
    @GetMapping("/home")
    public String homePage() {
        return "home"; // home.html
    }
}

// Returns JSON data
@RestController
public class ApiController {
    @GetMapping("/api/members")
    public List<Member> getMembers() {
        return memberService.getAll(); // Returns JSON
    }
}
```

c) @RequestBody vs @PathVariable

Annotation	Purpose	Location	Example
@RequestBody	Binds HTTP request body to Java object	Request body (JSON)	POST/PUT operations
@PathVariable	Extracts value from URL path	URL path	GET/DELETE by ID

Examples:

```
// @RequestBody - Data comes from JSON body
@PostMapping("/api/members")
public Member create(@RequestBody Member member) {
   return memberRepository.save(member);
}
// Request: POST /api/members
// Body: {"firstName": "Ali", "email": "ali@gmail.com"}

// @PathVariable - Data comes from URL
@GetMapping("/api/members/{id}")
public Member getById(@PathVariable Integer id) {
   return memberRepository.findById(id).orElseThrow();
}
// Request: GET /api/members/5
// id = 5 (extracted from URL)
```

SECTION C – ENTITY IMPLEMENTATION

Q3. Full Expense Entity Class

```java
package io.saadmughal.midterm.entities;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;

@Getter
@Setter
@Entity
@Table(name = "expenses")
public class Expense {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   @Column(name = "id", nullable = false)
```

```
    private Integer id;

    @Column(name = "date", length = 45)
    private String date;

    @Column(name = "amount", precision = 10)
    private BigDecimal amount;

    @Column(name = "description", length = 200)
    private String description;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "Members_id", nullable = false)
    private Member members;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "categories_id", nullable = false)
    private Category categories;

}
```

SECTION D - REPOSITORY & SERVICE LAYER

Q4. Repository Interfaces

1. MemberRepository

```
package io.saadmughal.midterm.repositories;

import io.saadmughal.midterm.entities.Member;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface MemberRepository extends JpaRepository<Member, Integer> {
}
```

2. CategoryRepository

```
package io.saadmughal.midterm.repositories;

import io.saadmughal.midterm.entities.Category;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface CategoryRepository extends JpaRepository<Category, Integer> {
}
```

3. ExpenseRepository

```
package io.saadmughal.midterm.repositories;
```

```java
import io.saadmughal.midterm.entities.Expense;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface ExpenseRepository extends JpaRepository<Expense, Integer> {
   List<Expense> findByMembers_Id(Integer id);
   List<Expense> findByCategories_Id(Integer id);
}
```

Q5. ExpenseService Class

```java
package io.saadmughal.midterm.service;

import io.saadmughal.midterm.entities.Expense;
import io.saadmughal.midterm.repositories.ExpenseRepository;
import io.saadmughal.midterm.repositories.MemberRepository;
import io.saadmughal.midterm.repositories.CategoryRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class ExpenseService {

   @Autowired
   private ExpenseRepository expenseRepository;

   @Autowired
   private MemberRepository memberRepository;

   @Autowired
   private CategoryRepository categoryRepository;

   // Get all expenses
   public List<Expense> getAllExpenses() {
       return expenseRepository.findAll();
   }

   // Add expense
   public Expense addExpense(Expense expense) {
       return expenseRepository.save(expense);
   }

   // Get expenses by category
   public List<Expense> getExpensesByCategory(int categoryId) {
       return expenseRepository.findByCategories_Id(categoryId);
   }

   // Get expenses by member
   public List<Expense> getExpensesByMember(int memberId) {
```

```
        return expenseRepository.findByMembers_Id(memberId);
    }

    // Update expense
    public Expense updateExpense(Integer id, Expense expenseDetails) {
        Expense expense = expenseRepository.findById(id).orElseThrow();
        expense.setDate(expenseDetails.getDate());
        expense.setAmount(expenseDetails.getAmount());
        expense.setDescription(expenseDetails.getDescription());
        expense.setMembers(expenseDetails.getMembers());
        expense.setCategories(expenseDetails.getCategories());
        return expenseRepository.save(expense);
    }

    // Delete expense
    public void deleteExpense(Integer id) {
        expenseRepository.deleteById(id);
    }
}
```

SECTION E - REST API DEVELOPMENT

Q6. ExpenseController Endpoints

```
package io.saadmughal.midterm.controller;

import io.saadmughal.midterm.service.ExpenseService;
import io.saadmughal.midterm.entities.Expense;
import io.saadmughal.midterm.entities.Member;
import io.saadmughal.midterm.entities.Category;
import io.saadmughal.midterm.repositories.MemberRepository;
import io.saadmughal.midterm.repositories.CategoryRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/expenses")
@CrossOrigin
public class ExpenseController {

    @Autowired
    private ExpenseService expenseService;

    @Autowired
    private MemberRepository memberRepository;

    @Autowired
    private CategoryRepository categoryRepository;
```

```
    // Get all expenses
    @GetMapping
    public List<Expense> getAllExpenses() {
        return expenseService.getAllExpenses();
    }

    // Get expenses by member
    @GetMapping("/member/{id}")
    public List<Expense> getExpensesByMember(@PathVariable Integer id) {
        return expenseService.getExpensesByMember(id);
    }

    // Get expenses by category
    @GetMapping("/category/{id}")
    public List<Expense> getExpensesByCategory(@PathVariable Integer id) {
        return expenseService.getExpensesByCategory(id);
    }

    // Add expense
    @PostMapping
    public Expense addExpense(@RequestBody Map<String, Object> payload) {

        Expense expense = new Expense();
        expense.setDate((String) payload.get("date"));
        expense.setAmount(new
java.math.BigDecimal(payload.get("amount").toString()));
        expense.setDescription((String) payload.get("description"));

        Integer memberId = (Integer) payload.get("memberId");
        Integer categoryId = (Integer) payload.get("categoryId");

        Member member = memberRepository.findById(memberId).orElseThrow();
        Category category =
categoryRepository.findById(categoryId).orElseThrow();

        // Match field names in Expense.java
        expense.setMembers(member);
        expense.setCategories(category);

        return expenseService.addExpense(expense);
    }

    // Update expense
    @PutMapping("/{id}")
    public Expense updateExpense(@PathVariable Integer id, @RequestBody
Map<String, Object> payload) {

        Expense expenseDetails = new Expense();
        expenseDetails.setDate((String) payload.get("date"));
        expenseDetails.setAmount(new
java.math.BigDecimal(payload.get("amount").toString()));
        expenseDetails.setDescription((String) payload.get("description"));

        Integer memberId = (Integer) payload.get("memberId");
        Integer categoryId = (Integer) payload.get("categoryId");
```

```
        Member member = memberRepository.findById(memberId).orElseThrow();
        Category category =
categoryRepository.findById(categoryId).orElseThrow();

        expenseDetails.setMembers(member);
        expenseDetails.setCategories(category);

        return expenseService.updateExpense(id, expenseDetails);
    }

    // Delete expense
    @DeleteMapping("/{id}")
    public void deleteExpense(@PathVariable Integer id) {
        expenseService.deleteExpense(id);
    }
}
```

Q7. (5 marks) Request JSON Body for Creating Expense

```
{
   "date": "2025-03-15",
   "amount": 1200,
   "description": "Team lunch",
   "memberId": 1,
   "categoryId": 1
}
```

Explanation:

- date: Expense date (String format)
- amount: Expense amount (Number)
- description: Details about the expense
- memberId: Foreign key reference to Member entity (ID = 1)
- categoryId: Foreign key reference to Category entity (ID = 1)

SECTION F - POSTMAN TESTING

Q8. API Calls + Payloads

Part (a): Create Member

Method: POST
URL: http://localhost:8080/api/members
Headers: Content-Type: application/json
Body:

```
{
   "firstName": "Ali",
   "email": "ali@gmail.com"
}
```

Expected Response:

```
{
   "id": 1,
   "firstName": "Ali",
   "email": "ali@gmail.com"
}
```

Part (b): Create Category

Method: POST
URL: http://localhost:8080/api/categories
Headers: Content-Type: application/json
Body:

```
{
   "name": "Food",
   "description": "Lunch, dinner, snacks"
}
```

Expected Response:

```
{
   "id": 1,
   "name": "Food",
   "description": "Lunch, dinner, snacks"
}
```

Part (c): Create Expense

Method: POST
URL: http://localhost:8080/api/expenses
Headers: Content-Type: application/json
Body:

```
{
   "date": "2025-03-15",
   "amount": 1200,
   "description": "Team lunch",
   "memberId": 1,
   "categoryId": 1
}
```

Expected Response:

```
{
   "id": 1,
   "date": "2025-03-15",
   "amount": 1200,
   "description": "Team lunch",
   "members": {
      "id": 1,
      "firstName": "Ali",
      "email": "ali@gmail.com"
   },
   "categories": {
      "id": 1,
      "name": "Food",
      "description": "Lunch, dinner, snacks"
   }
}
```

Part (d): Get All Expenses

Method: GET
URL: http://localhost:8080/api/expenses
Headers: None
Body: None
Expected Response:

```
[
  {
    "id": 1,
    "date": "2025-03-15",
    "amount": 1200,
    "description": "Team lunch",
    "members": {...},
    "categories": {...}
  }
]
```

Part (e): Get Expenses by Member

Method: GET
URL: http://localhost:8080/api/expenses/member/1
Headers: None
Body: None
Expected Response:


```
[
  {
    "id": 1,
    "date": "2025-03-15",
    "amount": 1200,
    "description": "Team lunch",
    "members": {
      "id": 1,
      "firstName": "Ali",
      "email": "ali@gmail.com"
    },
    "categories": {...}
  }
]
```

Part (f): Update Expense

Method: PUT
URL: http://localhost:8080/api/expenses/1
Headers: Content-Type: application/json
Body:

```
{
   "date": "2025-03-20",
   "amount": 1500,
   "description": "Updated expense",
   "memberId": 1,
   "categoryId": 1
}
```

Expected Response:

```
{
   "id": 1,
   "date": "2025-03-20",
   "amount": 1500,
   "description": "Updated expense",
   "members": {...},
   "categories": {...}
}
```

Part (g): Delete Expense

Method: DELETE
URL: http://localhost:8080/api/expenses/1
Headers: None
Body: None
Expected Response: 200 OK (No content returned)

APPENDIX: Complete Code Files

midtermapplication

```java
package io.saadmughal.midterm;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MidtermApplication {

   public static void main(String[] args) {
       SpringApplication.run(MidtermApplication.class, args);
```

```
    }

}
```

Pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.0.0</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>io.saadmughal</groupId>
    <artifactId>midterm</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>midterm</name>
    <description>midterm</description>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>17</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webflux</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
```

```
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webflux-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <configuration>
                    <annotationProcessorPaths>
                        <path>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </path>
                    </annotationProcessorPaths>
                </configuration>
            </plugin>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <excludes>
                        <exclude>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </exclude>
                    </excludes>
                </configuration>
            </plugin>
        </plugins>
    </build>
```

```
</project>
```

application.properties

```
# Database Configuration
spring.datasource.url=jdbc:mysql://localhost:3306/midterm_db?createDatabaseIfNo
tExist=true
spring.datasource.username=root
spring.datasource.password=batmanbinsuperman

# JPA/Hibernate Configuration
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
spring.jpa.properties.hibernate.format_sql=true

# Server Configuration
server.port=8080
```

Member Entity

```
package io.saadmughal.midterm.entities;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
@Entity
@Table(name = "members")
public class Member {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id", nullable = false)
    private Integer id;

    @Column(name = "first_name", length = 45)
    private String firstName;

    @Column(name = "email", length = 45)
    private String email;

}
```

Category Entity

```
package io.saadmughal.midterm.entities;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
```

```java
@Entity
@Table(name = "categories")
public class Category {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id", nullable = false)
    private Integer id;

    @Column(name = "name", length = 45)
    private String name;

    @Column(name = "description", length = 45)
    private String description;

}
```

Expense Entity

```java
package io.saadmughal.midterm.entities;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;

@Getter
@Setter
@Entity
@Table(name = "expenses")
public class Expense {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id", nullable = false)
    private Integer id;

    @Column(name = "date", length = 45)
    private String date;

    @Column(name = "amount", precision = 10)
    private BigDecimal amount;

    @Column(name = "description", length = 200)
    private String description;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "Members_id", nullable = false)
    private Member members;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "categories_id", nullable = false)
    private Category categories;

}
```

Category Controller

```
package io.saadmughal.midterm.controller;

import io.saadmughal.midterm.entities.Category;
import io.saadmughal.midterm.repositories.CategoryRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/categories")
@CrossOrigin
public class CategoryController {

    @Autowired
    private CategoryRepository categoryRepository;

    @GetMapping
    public List<Category> getAllCategories() {
        return categoryRepository.findAll();
    }

    @PostMapping
    public Category createCategory(@RequestBody Category category) {
        return categoryRepository.save(category);
    }

    @GetMapping("/{id}")
    public Category getCategoryById(@PathVariable Integer id) {
        return categoryRepository.findById(id).orElseThrow();
    }
}
```

Expense Controller

```
package io.saadmughal.midterm.controller;

import io.saadmughal.midterm.service.ExpenseService;
import io.saadmughal.midterm.entities.Expense;
import io.saadmughal.midterm.entities.Member;
import io.saadmughal.midterm.entities.Category;
import io.saadmughal.midterm.repositories.MemberRepository;
import io.saadmughal.midterm.repositories.CategoryRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/expenses")
@CrossOrigin
public class ExpenseController {

    @Autowired
    private ExpenseService expenseService;
```

```
    @Autowired
    private MemberRepository memberRepository;

    @Autowired
    private CategoryRepository categoryRepository;

    // Get all expenses
    @GetMapping
    public List<Expense> getAllExpenses() {
        return expenseService.getAllExpenses();
    }

    // Get expenses by member
    @GetMapping("/member/{id}")
    public List<Expense> getExpensesByMember(@PathVariable Integer id) {
        return expenseService.getExpensesByMember(id);
    }

    // Get expenses by category
    @GetMapping("/category/{id}")
    public List<Expense> getExpensesByCategory(@PathVariable Integer id) {
        return expenseService.getExpensesByCategory(id);
    }

    // Add expense
    @PostMapping
    public Expense addExpense(@RequestBody Map<String, Object> payload) {

        Expense expense = new Expense();
        expense.setDate((String) payload.get("date"));
        expense.setAmount(new
java.math.BigDecimal(payload.get("amount").toString()));
        expense.setDescription((String) payload.get("description"));

        Integer memberId = (Integer) payload.get("memberId");
        Integer categoryId = (Integer) payload.get("categoryId");

        Member member = memberRepository.findById(memberId).orElseThrow();
        Category category =
categoryRepository.findById(categoryId).orElseThrow();

        // Match field names in Expense.java
        expense.setMembers(member);
        expense.setCategories(category);

        return expenseService.addExpense(expense);
    }

    // Update expense
    @PutMapping("/{id}")
    public Expense updateExpense(@PathVariable Integer id, @RequestBody
Map<String, Object> payload) {

        Expense expenseDetails = new Expense();
        expenseDetails.setDate((String) payload.get("date"));
```

```
        expenseDetails.setAmount(new
java.math.BigDecimal(payload.get("amount").toString()));
        expenseDetails.setDescription((String) payload.get("description"));

        Integer memberId = (Integer) payload.get("memberId");
        Integer categoryId = (Integer) payload.get("categoryId");

        Member member = memberRepository.findById(memberId).orElseThrow();
        Category category =
categoryRepository.findById(categoryId).orElseThrow();

        expenseDetails.setMembers(member);
        expenseDetails.setCategories(category);

        return expenseService.updateExpense(id, expenseDetails);
    }

    // Delete expense
    @DeleteMapping("/{id}")
    public void deleteExpense(@PathVariable Integer id) {
        expenseService.deleteExpense(id);
    }
}
```

Member Controller

```
package io.saadmughal.midterm.controller;

import io.saadmughal.midterm.entities.Member;
import io.saadmughal.midterm.repositories.MemberRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/members")
@CrossOrigin
public class MemberController {

    @Autowired
    private MemberRepository memberRepository;

    @GetMapping
    public List<Member> getAllMembers() {
        return memberRepository.findAll();
    }

    @PostMapping
    public Member createMember(@RequestBody Member member) {
        return memberRepository.save(member);
    }

    @GetMapping("/{id}")
    public Member getMemberById(@PathVariable Integer id) {
        return memberRepository.findById(id).orElseThrow();
    }
}
```